

파이썬 비동기 프로그래밍 기초

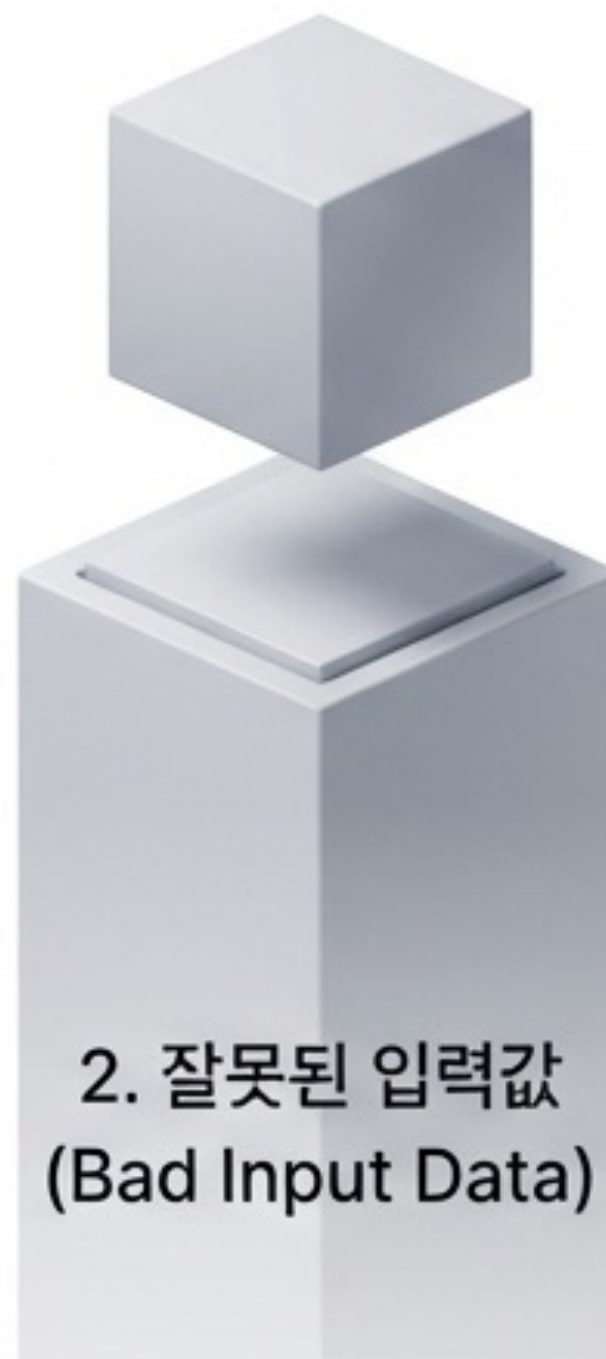
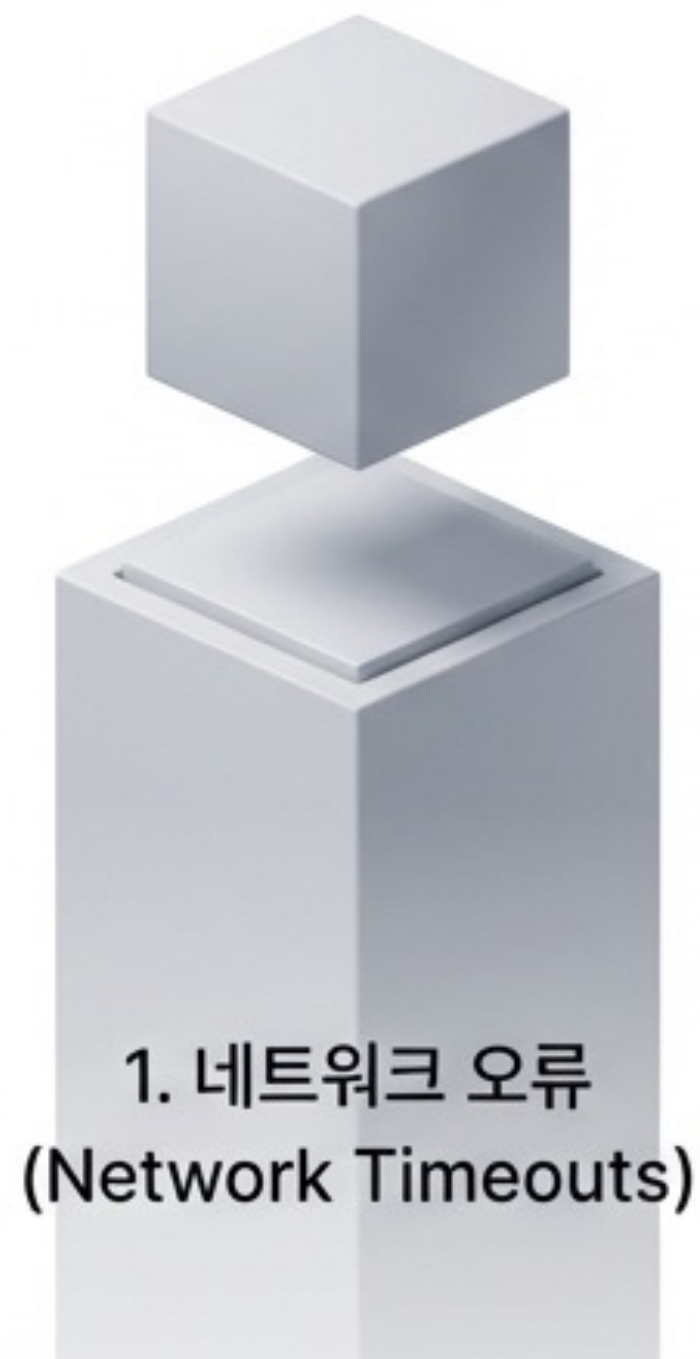
안정적인 비동기: 예외와 타임아웃

비동기 예외 처리



완벽한 코드는 존재하지 않는다.

실무의 환경에서는 모든 요청이 항상 성공한다고 가정할 수 없음
실패는 시스템의 결함이 아닌 예측해야 할 기본 변수





단일 코루틴 : 비동기도 기본 원리는 동일

단일 작업에서는 우리에게 익숙한 try/except가 동일하게 작동함

```
async def main():  
    try:  
        result = await risky_task('B')  
        print(result)  
    except ValueError as e:  
        print(f'예외 처리: {e}')
```

risky_task('B') 실행



ValueError('B 작업 실패') 발생



except 블록이 예외를 흡수하고
안전하게 종료

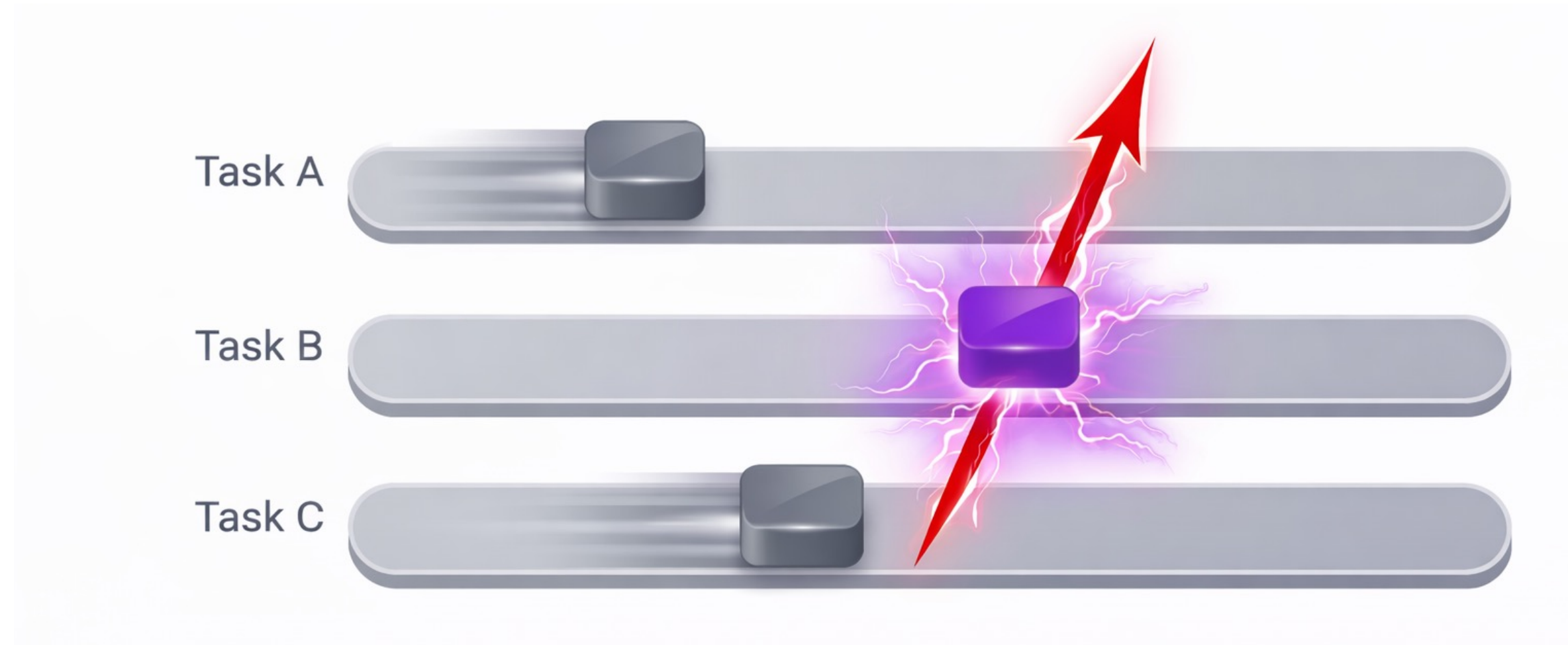


하지만 여러 작업을 동시에 실행(gather)한다면?

그 중 하나가 실패하면?



asyncio.gather의 기본 동작 : 예외 전파



예외 전파 (Exception Propagation)

기본 동작에서는 하나의 예외가 발생하면
즉시 호출자에게 전파되며, 나머지 태스크는 취소되지 않고 계속 실행됨



예외를 수집하는 옵션 (return_exceptions=True)

```
import asyncio

# ... (risky_task 정의) ...

results = await asyncio.gather(
    risky_task('A'),
    risky_task('B'),
    risky_task('C'),
    return_exceptions=True
)
```



결과 리스트
(Result List)

예외를 '반환 가능한 결과값'의 하나로 취급하도록 시스템의 규칙 변경



결과 리스트

'A done'

ValueError('B 작업 실패')

'C done'

작업별 상태 확인

각 작업의 성공/실패를 따로 확인 가능

선택적 재시도

실패한 작업만 골라 다시 실행 가능

부분 성공 활용

성공한 결과는 먼저 활용 가능



다중 코루틴 예외 처리 전략 비교

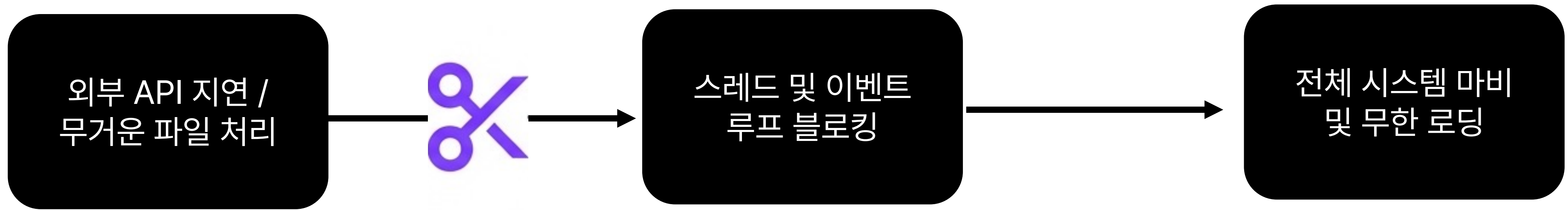
비교 기준	기본 <code>gather()</code>	<code>gather(return_exceptions=True)</code>	<code>TaskGroup()</code>
에러 발생 시 동작	첫 예외를 호출자에게 즉시 전파	예외를 결과 리스트에 포함해 반환	하나가 실패하면 남은 작업들을 취소하고 함께 정리
최종 반환값(Result)	예외 발생	[정상 데이터, Exception 객체, 정상 데이터] 혼합 리스트	결과 수집은 직접 관리
권장되는 실무 상황	하나의 실패를 바로 실패로 처리할 때	부분 성공/실패를 함께 수집할 때	작업 간 관련성이 높고, 실패 시 전체를 안전하게 정리해야 할 때

비동기 time out 설정



비동기의 함정 : 무한정 기다리기?

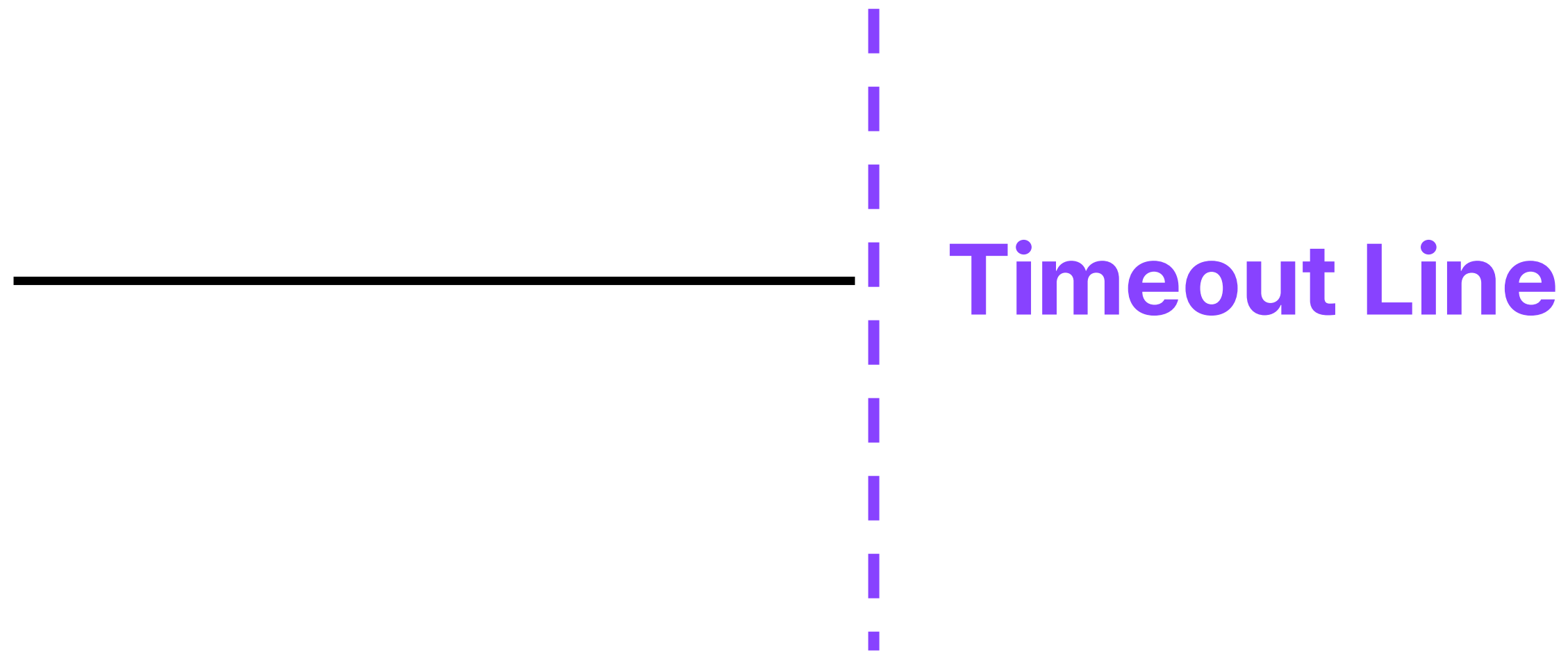
예상치 못한 비정상적인 지연 발생



하나의 지연이 전체 흐름에 영향을 미침



한계선 긋기 : Timeout



이 작업을 언제까지 기다릴 것인가?

프로그램이 끝없이 기다리지 않고, 일정 시간 이후에 다른 대응을 할 수 있도록 제한을 두는 것



구현 : `asyncio.wait_for()`

- 느린 작업: `slow_task()`는 5초 대기를 시뮬레이션
- 차단기 장착: `asyncio.wait_for()`를 사용하여 해당 작업을 감싸고, `timeout=2.0`으로 2초의 제한을 둠

```
import asyncio

async def slow_task():
    await asyncio.sleep(5) # 5초 소요되는 작업
    return "done"

async def main():
    # 2초의 한계선 설정
    result = await asyncio.wait_for(slow_task(),
                                    timeout=2.0)
```



asyncio.wait_for() 문법 상세

```
import asyncio

async def slow_task():
    await asyncio.sleep(5) # 5초 걸리는 작업
    return "done"

async def main():
    try:
        result = await asyncio.wait_for(
            slow_task(),
            timeout=2.0)
    except asyncio.TimeoutError:
        print("시간 초과!")
```

```
await asyncio.wait_for(
    코루틴, timeout=초)
```

인자: **코루틴** + **timeout** (초 단위)

반환값: 코루틴의 **실행 결과값**

초과 시: **asyncio.TimeoutError** 발생

핵심 포인트

- 시간 내 완료 → **정상 결과 반환**
- 시간 초과 → 태스크 **자동 취소** + TimeoutError
- try/except로 **안전하게 처리**하는 것이 필수



TimeoutError 처리

```
async def main():  
    try:  
        result = await asyncio.wait_for(slow_task(), timeout=2.0)  
        print(result)  
    except asyncio.TimeoutError:  
        print("timeout 발생") # 다른 대응 시작  
  
asyncio.run(main())
```

2초 안에 끝나지 않으면 TimeoutError가 발생하며,

여기서 프로그램이 멈추지 않고 안전하게 다음 대응(예: 재시도, 에러 메시지 반환)을 할 수 있음



정상 완료 vs. 시간 초과

조건 A (제한 시간 > 작업 시간)	조건 B (제한 시간 < 작업 시간)
상황 : 예) 제한 5초, 작업 2초	상황 : 예) 제한 2초, 작업 5초
결과 : 정상 반환 ("done")	결과 : TimeoutError 발생
상태: 성공 (Success)	상태: 예외 처리 진입 (방어 성공)



실무에서 Timeout이 필수적인 이유

[Timeout이 없을 때]



전체 시스템 흐름 꼬임 / 사용자 경험 악화

[Timeout이 있을 때]



빠른 피드백 제공 / 시스템 리소스 보호